Help

```
:help script               Scripting help
:help eval                 Main reference
:help expression-commands  Build-in commands, including let, unlet

:help Variable             Data types help
:help Dictionary
:help Lists
```

==========

======

Basics

Syntax

```
" Multiple lines
" Use '\'
mycommand arg1        " Multiline command
     \arg2
     \arg3
let a="Hello          " Multiline string. Need \n for new line.
\ World\n
\ A\n
\ B\n"
```

Variables

```
:help let             " help on 'let'
let a = 1
let a=&fietype        " option value
let a=$MYVIMRC        " read shell value
let a=@a              " 'a' registry value
let b = a             " Dereference a variable
let &filetype=a       " Set option
let $MYVIMRC=a        " Set shell variable
let b = exists("a")   " Result of function
let b = system("ls")  " Result of shell command.

let a += 1            " a = a + 1
let a -= 1            " a = a - 1
let b .= 'abc'        " b = b . "abc"

unlet a               " delete. Error if the variable doesn't
exist.
unlet! a              " delete. No error.

exists("a")           " 0 if variable "a" exists. 1 otherwise
exists("*myFun")      " function myFun exists.
exists(":MyCmd")      " command MyCmd

echo a                " Print a
echo exists("a")


" Variable types
help internal-variables
let g:a=1             " global
let w:a=1             " window
let t:a=1             " tab
let b:a=1             " buffer
let l:a=1             " local to function
let s:a=1             " local to script
let a:a=1             " function argument
```

```
let v:a=1             " predefined by  vim
echo v:a


"" Expressions
"
g:a             " Variable
$USER           " shell variable
&tabstop        " options
@0              " register 0

" eval()
echo eval("a")                " Evaluate variable "a"
echo eval("$USER")
echo eval("&tabstop")
echo eval("@0")


"" Operations
"
" - For boolean operations
"   true = 1
"   false = 0
"
:help expressions-syntax
echo 1+2
echo 1*2
echo 4/2
echo 3%2
echo a > b ? a : b   " conditional
echo a != b          " 1 if true, 0 if false
echo a == b
echo a >= b
echo a > b
echo "ab" . "cd"     " concat string
echo !x              " NOT x
echo x && y          " x AND y
echo x || y
```

Data types

```
:help variables
1               " Number
1.0             " Float
"abc"           " String
'abc'
"\""            " '"' characater. '\' for escape
"\\"            " '\' character
[1, 2, 3]       " List. See below.
{"name": "Yui"} " Dictionary. See below.


" Coercion
1 + 1.0          " 2.0. Coerced to float
1.0 + 1          " Same
1 + '1'          " 2. Coerced to number
1 + 'abc'        " 1. Ignored if nonsense
1 . 'abc'        " "1abc". Coerced to string

" Conversions
str2float("1.1")     " 1.1
str2nr("1")          " 1
float2nr(1.0)        " 1
float2nr(1.1)        " 1. Cutoff
```

```
nr2char(97)          " 'a'. Character code to char
```

Flow control

```
" Single line
echo "a" | echo "b"


" expr ? expr1 : expr2
"
echo x > y ? 'a' : 'b'
x > y ? echo 'a' : echo 'b'     " ERROR. echo 'a' is not an
expression


"
" Conditions
"
" 0 = false
" non-zero = true
if a == b
     ...
elseif a == c
     ...
else
     ...
endif

" check option value
if &filetype == "python"
     …
" call function
if checkTrue()
     …
" single line
if a == b | echo "True" | else | echo "False" | endif


"" Loops
"
" Basic
for  i in [1, 2, 3]
     echo i
     "
     break
     "
     continue
endfor
" With range()
for i in range(1,5)
     echo i             " i = 1, …, 5
endfor
" While loop
let i = 0
while i < 10
     let i += 1
endwhile


"" list
let mylist = [1,2]
for item in mylist
 …
endfor
```

```
" with index
for i in range(0, len(mylist)-1)
 let item = mylist[i]
 …
endfor
" with while
let i = 0
while i < len(mylist)
 let item = mylist[i]
 let i = i + 1
endwhile
```

Functions

```
"" Help
help :function
help function-argument


"" Calling
"
call MyFunction()
let a=MyFunction()           " Set to variable
echo MyFunction()            " Print


"" Defining
"
function MyFunction(x, y)
     " local variables with a:
     echo a:x a:y
     …
     return 10
endfunction

" Optional variables
function MyFunction2(x,…)
      a:0       " Number of optional arguments.
      a:1       " First optional argument
      a:000[0]  " Same
      a:2       " Second optional argument
      a:000[1]
endfunction

call MyFunction2(10,11,12)     " x=10,a:0=2,a:000=[11,12]

" abort: exit immediately on error
function MyFunction() abort
     gibberish
     echo "Never reaches here"
endfunction


" Return values
function MyFunction()
     ...
     return 10

     return          " no return  value
     return 0        " same
endfunction
```

```
" Redefine
function! MyFunction(…)        " Overwrite previous definition
endfunction

function MyFunction()          " ERROR if already defined
endfunction


" Delete
delfunction MyFunction
```

Strings

```
"" Basics
let x = "abc"
x[0]          " 'a'
x[0:1]        " 'ab'. i.e. substring
x[100]        " empty string


"" Functions
:help functions
strlen("abc")          " 3
join(["a","b"],'_')    " 'a_b'
split("a_b", "_")      " ["a","b"]

" substitute(VALUE, PATTERN, REPLACEMENT, FLAGS)
substitute("abc", "[a-z]", "1", "")
```

Lists

```
:help List

let x = [1,2,3]
let x = [1, 'a', 'b']               " Can be mixed


" Concatenate
let y = x + [4,5]
let x += [4,5]

" Comparison
let y = x           " y and x refer to the same list, so changing
one also changes the other.
let z = [1,2,3]
echo x == z              " 1. Compares list by value.
echo x is z              " 0. Checks if they are the exact same
list instance.

let list = [1,2,3]
list[0]             " First item
list[1:10]          " Copy of list subset [list[1],…,list[10]]
list[1:]            " To end of list. [list[1],… ]
list[1:-1]          " same
list[:3]            " From beginning. […,list[3]]
list[0:3]           " Same
list[:]             " Copy of whole list

" List operations
call insert(list, 1)          " prepend
call add(list, 1)             " append
call extend(list, [2, 3])     " concat
let l = remove(list, 3)       " remove list[3]
```

```
unlet list[3]                  " same
let l = remove(list, 3, -1)    " remove list[3, …]
unlet list[3: ]                " same

" list string operations
let s = join(list, ',')
let list = split("a b c")
```

Dictionary

```
:help Dictionary

let x = {}                 " Empty
let x = { "value": 1 }
let x = {                  " multiline
\ "value": 1,
\ "name": "Yui"
\}

has_key(x, "name")    " 1
has_key(x, "aaa")     " 0
x["value"]            " 1
x["name"]             " Yui
```